

Click-to-Source 3D

Auto-Instrumentation — Experimental Results

Five Hypotheses, Tested Against Ground Truth

1. Executive Summary

Six mechanisms for automatic provenance capture were designed as experiments and run against a real procedural generator, with hand-verified ground truth from Stage 4 as the oracle. All six produced decisive results.

The first five established a finding that held without exception:

Anything recording before the destination exists cannot route.

Four of those five capture runtime arguments — two perfectly, with a third recovering named parameters. None can determine which mesh and which instance index a captured value belongs to. The one that routes does so natively, at 100%, because it records where mesh identity is the call's own receiver — and it captures no arguments at all.

Capture quality and routing are independent axes. No single mechanism occupies both.

H6 tested whether they compose, and they do. Chaining transform capture, `Matrix4.prototype.clone()` interception, and destination interception produces `{mesh, instanceId, args}` for every instance of both components, with zero manual annotation, byte-identical to ground truth — verified in Node and again in the live browser. It also recovers global generation order, which H4 explicitly could not.

	Argument capture	Routing
H1 — Wrapper API	Best: named parameters, 200/200	Zero — 0/200 records touch a mesh
H2 — Call-site rewriting	Positional, 200/200	Contingent on a glob accident
H3 — Deterministic replay	Exact, byte-identical	Fails — recovers values, not destination
H4 — <code>setMatrixAt</code> interception	None	100%, structural — 1205/1205 live
H5 — Dummy proxying	Exact, byte-identical	Provably impossible
H6 — The composition	Exact, byte-identical	1205/1205 live, plus global order

The scope of that result needs stating precisely: **hand-rolled instancing is solved end to end.**

Library-owned instancing is not — under drei `<Instances>` there is no `setMatrixAt` call anywhere in the program, so the join has no destination side to attach to. These remain two separate claims.

2. Method

2.1 Ground truth

Stage 4 tagged four objects by hand and verified them in a running browser. `Trees.jsx`'s fully-built `instanceSourceRefs` array was captured once from the live application via a temporary hook, then reverted, producing `h3/node200.json` — 61,732 bytes, SHA-256 `c569dd23...c34b2893`, 200 instances across colour groups `[198, 2, 0, 0]`.

Every experiment compared against that file. Most compared full-payload hashes rather than sampling, so "zero divergence" is a complete claim rather than an inference from spot checks.

2.2 Discipline

Each experiment ran in an isolated scratch folder outside both repositories. Prior experiments' folders were read-only throughout — verified by unchanged modification times. The `LowPolyWorldGen` repository was verified untouched at the end of every experiment: `git status --porcelain --untracked-files=all` empty, HEAD unchanged at `7c4a956`.

Two experiments required a temporary hook inside the application's own module graph to observe the real React pipeline. Both showed the diff before applying, applied it, captured, reverted, and verified no residue remained.

2.3 A note on the tree count

The browser renders 200 trees, not 260. `Scene.jsx` sets `treeCount = [140, 200, 260][perfTier]`, and `perfTier` is driven by drei's `PerformanceMonitor` — it sat at tier 1 throughout observation and never inclined. Tree count is performance-derived, not seed-derived. This affects instance counts but not any conclusion below.

3. H3 — Deterministic Replay

Premise: generation is seeded and deterministic, so provenance need not be stored. Store the seed, generator identity, and instance index; recompute arguments on click.

3.1 Determinism — confirmed

Three separate Node processes, seed 42, count 260, importing the real `TerrainGenerator`, `mulberry32`, and `ALPINE_TERRAIN` modules rather than copies:

PID	bytes	SHA-256
9488	80,134	<code>c238c9c9...000d3c0f</code>
10776	80,134	<code>c238c9c9...000d3c0f</code>
9304	80,134	<code>c238c9c9...000d3c0f</code>

Byte-identical across processes. Loop statistics: 3,005 iterations, 7,308 RNG draws, colour groups `[255, 5, 0, 0]`.

3.2 Replay matches the browser exactly

	bytes	SHA-256
Browser (ground truth)	61,732	c569dd23...c34b2893
Node replay, count 200	61,732	c569dd23...c34b2893

Zero divergence across all 200 instances and all four colour groups. No accumulated RNG drift — the last instance is as exact as the first.

3.3 Why it fails: you cannot jump to instance N

The placement loop rejects candidates on four paths, and the two jitter draws happen before any of them. Rejection accounting for the 260-run, which closes exactly:

Outcome	Count	Draws each	Total
Rejected at suitability < 0.08	2,227	2	4,454
Rejected probabilistically	374	3	1,122
Rejected by ledge check	144	3	432
Rejected by slope check	0	3	0
Placed	260	5	1,300
			7,308

3,005 iterations produce 260 instances, and the stride is data-dependent: gaps between consecutive placements span **1 to 140 iterations** (53 distinct values) and **5 to 294 RNG draws** (69 distinct values). Which gap applies depends on terrain sampled at each rejected point. The RNG offset for instance N is knowable only by having already walked iterations 1 through N.

3.4 Cost

Cold start, median of three, early-exiting the moment the target is built:

Target instance	Loop only	Full process wall	Iterations	sample() calls
0	4.4 ms	330 ms	12	41
130	66.1 ms	384 ms	1,970	7,868
259	92.4 ms	415 ms	3,005	13,513

Warm — modules loaded, generator reused, JIT hot — the full 260-instance loop runs in **32 ms median**. Fixed overhead is Node boot $\approx$ 110 ms plus module import 57 ms.

Cost is dominated by `generator.sample()`: recovering instance 259 requires 13,513 calls. Each iteration costs five samples merely to fail the suitability gate.

3.5 Verdict

The premise holds — generation is deterministic and replay is exact. But cost is linear in *iteration* index, not instance index, and the two relate only through terrain data. At ~415 ms cold for the last instance, this is not viable as a click-time mechanism on hardware already running `AdaptiveDpr` to hold framerate.

A second blocker: a click yields an `instanceId` within one mesh. For the trunk mesh that is the global placement index, but for foliage it is an index within a colour group, so replay must also reproduce the `colorIdx` bucketing to establish the mapping.

Viable offline. Not viable at click time.

4. H5 — Dummy Proxying

Premise: R3F instancing follows one idiom — a shared `dummy = new THREE.Object3D()`, set transform, `updateMatrix()`, clone. Proxy the dummy, record state at each `updateMatrix()`, and the nth recording is the nth matrix pushed.

4.1 Transforms — exact

Run	SHA-256	vs browser
Browser (H3 capture)	<code>c569dd23...c34b2893</code>	—
Plain <code>Object3D</code> replay	<code>c569dd23...c34b2893</code>	match
Proxied dummy replay	<code>c569dd23...c34b2893</code>	match

Zero matrix element differences across 200×16 elements — the proxy perturbs nothing. Rebuilding the payload from the recordings alone, using nothing from the loop, produced `c1cbd6c7...cd1f8bc4e` against a ground truth of `c1cbd6c7...cd1f8bc4e`: byte-identical, zero divergent instances.

The recording is strictly richer than the five args currently stored — `rotation.x` and `rotation.z` are always zero, Euler order always `XYZ`, scale always uniform.

4.2 Routing — provably impossible

Recorded fields are `["position", "rotation", "order", "scale"]`. **No material, no colour, no geometry, no mesh.**

For Trees, `colorIdx` happens to be a pure function of `height`, which is `position.y`, so it can be re-derived — 0 mismatches, closest bucket-boundary margin 0.029590. But that means re-implementing `Trees.jsx:151-169`, constants included. The mapping is relocated, not dissolved.

Skipping the derivation and assuming `foliage instanceId j === recording j`:

Group	Instances	Resolve to the wrong tree
0 (count 200)	198	21 (10.6%)
1 (count 200)	2	2 (100%)
0 (count 260)	255	78 (30.6%)

1 (count 260)	5	5 (100%)
---------------	---	----------

Group 0 is *nearly* identity, which is the dangerous part — it would pass casual testing and be wrong 10–30% of the time.

4.3 GroundCover — the proof of absence

Five meshes, two entity types, 805 recordings (750 grass + 55 bush), 1:1 with matrices, zero element differences.

`grassVariant` and `bushVariant` come from `rng()` draws that never touch the transform. Re-running with altered variant thresholds — same seed, same draws:

```
recordings identical : true      (sha 284a95b921d7b752 on both runs)
grass mesh sizes : [262,263,225] -> [413,179,158]
bush mesh sizes : [31,24]       -> [10,45]
instances that moved to a different mesh: 239 (29.7%)
```

Byte-identical recordings, 29.7% of instances in a different mesh. The recording provably cannot distinguish the two worlds. This is a proof of absence, not a failure to find.

No heuristic rescues it: using `scale.x / scale.z`, grass spans 0.8811–1.1385 and bush spans 0.8374–1.3525, with 750/750 grass and 31/55 bush inside the overlap band. The best single-threshold split still misclassifies 33 of 805.

4.4 What defeats it

Nine deviation cases, each running the same proxy over a loop differing in exactly one way:

Pattern	Result
Two dummies, one proxied	Breaks silently — nth recording is instance 2n
Direct <code>Matrix4</code> , no <code>Object3D</code>	Breaks — 0 recordings, no warning
<code>matrix.compose()</code> instead of <code>updateMatrix()</code>	Breaks — 0 recordings, same operation unhooked
<code>setMatrixAt</code> from a separate cursor	Breaks — recordings right, index wrong
<code>updateMatrix()</code> then instance rejected	Breaks — off-by-N from one stray call
<code>updateMatrix()</code> twice per instance	Breaks — 10 recordings, 5 matrices
Three's internal <code>updateMatrixWorld()</code>	Survives — internal calls bypass the proxy
Proxy identity	Survives — <code>instanceof</code> and <code>isObject3D</code> hold
<code>push(matrix)</code> without <code>.clone()</code>	Survives — and would expose the bug

One of these already ships. `GroundCover.jsx:147-165` builds 19 placement matrices with `new THREE.Matrix4().makeScale().premultiply(...)` and no `Object3D` anywhere. It is geometry assembly rather than instancing, so it does not corrupt the mapping — but it is a live instance of the pattern the recorder is blind to.

Verdict: solves transforms, fails routing, and the failure is structural.

5. H4 — setMatrixAt Interception

Premise: maintain a shadow stack; instrumented call sites push a provenance frame, and anything written while a frame is live is attributed to it. Pair with intercepting `InstancedMesh.prototype.setMatrixAt`.

5.1 The frame mechanism failed completely

The loop from `Trees.jsx:104-201` was reproduced verbatim with `pushFrame()` at the top of the body and `popFrame()` at the bottom. Both phases shared one clock:

```
lastMatrixBuilt_t: 2844   loopEnd_t: 2846   firstSetMatrixAt_t: 2847
writesInterleavedWithLoop: 0
matricesBuiltBeforeFirstWrite: 200          (all of them)

applicationWrites: 400
writesWithLiveFrame: 0   writesWithNullFrame: 400
attributionRate: 0.00%
```

Zero of 400 writes attributable. The generous control — push but never pop, so a frame is always live — pinned every write to the final loop iteration and was correct once, by coincidence: 1/200, or 0.5%.

`useMemo` runs in the render phase; `useEffect` runs in the commit phase. No synchronous frame survives React's commit. The mechanism is not weak here; it is inapplicable.

5.2 Interception carries mesh identity natively

The hypothesis failed. The experiment succeeded, by a mechanism the hypothesis never proposed.

Total calls intercepted	800
— constructor identity fill	400
— application writes	400
Writes with mesh identity + instanceId	400 / 400 (100%)
Distinct meshes resolved	3

`this` is the destination mesh; `index` is the destination slot. Both are the call's own receiver and arguments. Nothing is derived.

Recovering `colorIdx` without `position.y`. `Trees.jsx` pushes the same `dummy.matrix.clone()` into both the trunk array and the colour-group array, so the trunk stream — where `index` is generation order — joins to the foliage streams on exact matrix bytes:

```
trunkMatrixKeyCollisions: 0
foliageWritesJoined: 200   unjoined: 0
colorIdx    correct 200 / 200  -> 100.00%
indexInMesh correct 200 / 200  -> 100.00%
usedPositionY: false
```

Control — strip the receiver, leaving H5's information: `indicesReusedAcrossMeshes: 198`, `canNameDestinationMesh: false`. Every mesh restarts its index at 0.

5.3 An unbudgeted finding

Three r173's `InstancedMesh` constructor runs `for (i < count) this.setMatrixAt(i, _identity)`. A prototype patch therefore sees **2× traffic**, half of it meaningless identity fill. The only discriminator at the interception point is object identity of the matrix argument — the constructor passes one shared `_identity` singleton, application writes pass distinct clones. That separates 400/400 cleanly, but it is a real trap.

5.4 GroundCover — and the symmetric traps

```
distinctMeshUuids: 5   distinctMaterialObjects: 2   distinctGeometryObjects: 5
materialSeparatesGrassFromBush: true
materialSeparatesVariants:      false   (all 3 grass meshes share ONE material)
geometrySeparatesVariants:      true
```

```
(kind, variant, instanceId) recovery: 805 / 805  -> 100.00%
usedPositionY: false
```

Trees' foliage meshes share one geometry and separate only by material. GroundCover's grass variants share one material and separate only by geometry. **An implementation keying on only one of those will work on one component and silently fail on the other.**

The variant-threshold control, run against both approaches:

Recorder	Default	Altered	Verdict
H5 proxy recording	284a95b921d7b752	284a95b921d7b752	Byte-identical — blind
H4 intercepted stream	0fd95bc328e7d984	fb8d8512f6a42993	Differs

36.89% of instances relocated. Interception carries information the proxy provably lacked.

Limit: Trees has a trunk mesh holding every instance in generation order. GroundCover has no such mesh — the five streams are a partition, and the grass/bush interleave is destroyed before the first write. Interception recovers `(kind, variant, instanceId)` completely; it does not recover global generation order.

5.5 Cost and blast radius

Median of seven trials, 200,000 calls each:

	ns/call	vs baseline
Unpatched	31.3	1.00×
Patched, no capture	63.2	2.02×
Minimal capture (uuid + index)	88.0	2.81×
Full capture (<code>matrix.toArray</code> , <code>WeakMap</code>)	1116.1	35.66×

On the real workload — 1,205 application writes, 2,410 intercepted calls per scene build — that is **2.6 ms** full capture, **0.14 ms** minimal, one-off at mount.

The patch is process-global. Every `InstancedMesh` in the process pays the call-through, including drei's `Cloud`, `Sampler`, `Instances`, and `Outlines`.

What defeats it: direct `instanceMatrix.array` writes (0 intercepted, instance silently changed), out-of-order or sparse `setMatrixAt`, and `count` exceeding instances written. None appear in `src/`. The project's only pattern is **write-once at mount**, which is the sharp constraint: the probe must be installed before the first commit, because there is no replay.

5.6 Browser confirmation

A temporary probe was applied to the application's own module graph, captured, and reverted. **Every Node number reproduced exactly.**

```
setMatrixAt calls intercepted: 2410
  constructor identity fill: 1205
  application writes: 1205    ( = 400 Trees + 805 GroundCover )
updateMatrix calls: 2149
```

```
updateMatrixCallsBeforeFirstApplicationWrite: 2149
applicationWritesBeforeLastUpdateMatrix: 0
shadowStack: writesWithLiveFrame 0 / writesWithNullFrame 1205
```

1,205/1,205 writes carried mesh and instanceId across 8 meshes, every one contiguous. Foliage distribution `[198, 2, 0, 0]` — identical to ground truth.

Two corrections from the live run: `StrictMode` does *not* double the writes — exactly one write pass was measured. And `userData.instanceSourceRefs` is `null` at write time on all 8 meshes, because the effect assigns it after the `forEach`.

Verdict: routes structurally. The only hypothesis that does.

6. H2 — Call-Site Rewriting

Premise: a Babel transform rewrites call sites rather than declarations, so directory-based opt-in gains full argument capture with zero developer annotation — reopening the Stage 2 decision believed settled.

6.1 The transform works

`Terrain.jsx` has 31 call sites; one was rewritten. Position, `terrainMasks`, and normal buffer hashes were all identical between transformed and untransformed arms, with arguments `[42, {worldSize: 400}]` captured. Rewriting the hot path — `generator.sample(x, z)` — captured **58,081 invocations** with real arguments and return values, geometry still byte-identical.

6.2 It reaches loop bodies further than expected

`Trees.jsx` has 69 call sites, 37 inside the placement loop. Twelve are the existing hand-written provenance block and were excluded, leaving **25 usable sites**. Run: 12,755 events over 2,445 iterations producing 200 placements, reproducing ground truth exactly.

Reconstruction from intercepted calls alone: 200/200 exact on all five args, plus `colorIdx` and a contiguous within-group index.

Two qualifications:

It works because every value crosses a call boundary — a property of this code, not a guarantee.

Crosses a call boundary	Does not
<code>jx, jz, height, finalScale, yaw, suitability, slope, altNorm, colorIdx, maxDrop, hN/hS/hE/hW</code>	<code>gx, gz, altitudeFactor, suitabilityFactor, baseScale</code>

Plain arithmetic emits no call and is invisible. `finalScale` is recoverable only because `dummy.scale.setScalar(finalScale)` happens to be a call. Three.js's setter-heavy style is what rescues this.

The transform recovers values, not names. `position.set(a, b, c)` yields `arg0/arg1/arg2`.

6.3 Routing succeeded by accident — and the experiment proved it

Loop-time events occupy ticks 32–12,786; every `setMatrixAt` occupies 12,795–13,200. Zero loop-time events reference an `InstancedMesh`. Recording at the call is recording before the destination exists.

But the glob is file-scoped, so it *also* rewrites `meshRef.current.setMatrixAt(i, m)` at `Trees.jsx:251` — whose receiver is the mesh. Three scopes, measured:

Scope	Captures with an instance index	Mesh references	Routes
A — whole file	400	3 meshes	Yes
B — L251 excluded	0	3, via bounding calls only	No
C — library-owned instancing	0	0	No

Confirmed in the real browser against the application's own `instanceSourceRefs`: **400/400 agree** in scope A.

When H2 routes, it routes *because it is doing what H4 does* — recording at the destination. The loop-time captures never route on their own.

6.4 Costs

Bundle (real production build, `src/components/**`): baseline 1,757,101 B → instrumented 1,796,469 B, **+39,368 B (+2.24%)**; gzip +5.42 kB (+1.11%). 369 meta objects survive minification at ~107 B per site.

Tree-shaking: a real regression. An unused exported module-level value initializer is eliminated at baseline (1,453 B) but survives instrumented (2,632 B) as a bare statement, because the wrapper is opaque and potentially side-effecting. Unused *function declarations* still shake correctly. The regression is specific to module-level value initializers — precisely the shape of `FOLIAGE_COLORS` and `TRUNK_COLOR`.

Runtime volume: one page load produced **2,316,657 captured events** from 364 static sites — Terrain 2,066,074, GroundCover 219,630, Trees 30,819.

HMR: survives, accumulates, never invalidates.

	Placements	setMatrixAt	Meshes written	Detached from scene
--	------------	-------------	----------------	---------------------

Clean load	400	400	3	0
+ comment-only edit	800	800	6	3
+ value change	1,200	1,200	9	6

Each generation is internally consistent, but the log grows without bound, two-thirds points at dead meshes, and nothing marks which generation is current.

StrictMode asymmetry: loop-time captures double (400 `position.set` for 200 trees) while destination writes stay single. An independent argument for destination-side recording.

False positives: 364 call sites rewritten by a naive `src/components/**` glob, of which 81 (22%) are generator-relevant. **283 (78%) are not.** Top noise: `rng` ×22, `Math.sin` ×11, `Number` ×11, `useMemo` ×10.

Verdict: refutes the reason the Stage 2 decision was closed; the decision survives on better grounds.

7. H1 — Wrapper API

Premise: the incumbent candidate, and the only one argued to carry argument capture and destination identity in the same construct.

7.1 The refactor cost

`Trees.jsx` has no function to wrap. Extracting the loop body empirically — re-running until it stopped throwing `ReferenceError` — revealed the boundary:

```
free names that MUST be supplied (11): gx, rng, spacing, gz, generator,
    numColorGroups, dummy, colorGroupMatrices, colorGroupSourceRefs,
    trunkMatrices, trunkSourceRefs
```

```
loop variables : 2      closure state : 5      output sinks : 4
`continue` statements that had to change form : 4 -> `return null`
```

Eleven free names, or seven if the sinks become a return value. The four `continue` paths become `return null`; `break outerLoop` never crosses the boundary.

The finding that shapes everything after it: none of `x`, `z`, `height`, `scale`, `yaw` appear in that list. All five are computed inside the body. At the natural extraction boundary they are results, not parameters.

Two boundaries were therefore built — **Refactor A** (whole body, `createTree(gx, gz, ctx)`) and **Refactor B** (`createTree(x, z, height, scale, yaw)`). Both are byte-identical to ground truth before any wrapping. Churn: A is +72/−57 code lines; B is +18/−13.

Is the result better code? A is worse: the 5-field `ctx` bag exists only to feed the extraction, and `dummy` — a shared mutable `Object3D` — now crosses a function boundary, which is exactly the aliasing an extraction should eliminate. B is neutral to slightly better, but earns it by demoting itself: all the interesting work stays in the loop and the extracted function is five lines of `Object3D` manipulation.

Neither is a refactor anyone would do for its own sake.

7.2 Argument capture — the best of the five

```
src-wrapB
  wrapper records          : 200
  recovered parameter names : ["x","z","height","scale","yaw"]
  of {x,z,height,scale,yaw} present in the arg list: 5/5
  all five exact           : 200 / 200
```

Real names, from `Function.prototype.toString()`. Genuinely better than H2's positional labels.

Two qualifications:

Refactor A also recovers 200/200 — but not from arguments. It decomposes the returned `Matrix4` back into position, scale, and yaw. That is H3-style reconstruction wearing a wrapper. A also records 2,445 calls for 200 instances — a 12.2× over-capture, because the wrapper sits outside the four rejection gates.

The names do not survive the build.

```
src-wrapB  minify=false  paramNames=["x","z","height","scale","yaw"]  fn.name="createTree2"
src-wrapB  minify=true   paramNames=["i","n","o","e","a"]             fn.name=""
```

Minified, `["i","n","o","e","a"]` is worse than `arg0/arg1/arg2` — H2's labels are honestly positional; these look like names and are noise.

7.3 Routing — zero

```
wrapper records whose `this` is an InstancedMesh : 0 / 200
wrapper records with a mesh anywhere in arguments : 0 / 200
wrapper records with a mesh in the return value   : 0 / 200
InstancedMesh objects in existence at each call   : max 0, min 0
first mesh constructed at tick : 401    last wrapper call at tick : 200
```

At every one of the 200 calls, **not a single `InstancedMesh` exists anywhere in the program yet.** Not weak access — zero.

Pairing does succeed when an H4-shaped hook is present: 400/400 joined by matrix object identity, 200/200 args matching ground truth. But the audit trail is explicit — the wrapper contributed the join key; `setMatrixAt` contributed (`mesh`, `index`). Wrapper alone: 0/200 mesh references, 0/200 instance indices, routing NO.

7.4 Scope C is worse than a scoping problem

Scope	Args	Routes
A — destination hook in scope	5/5	200/200
B — destination out of scope	5/5	0/200
C — library-owned instancing (drei <code><Instances></code>)	5/5	0/200

In the installed drei 10.x, `core/Instances.js:150` writes transforms as `instanceMatrix.toArray(matrices, i * 16)` into a `Float32Array` inside `useFrame`. Grepping the real package's ESM build, `setMatrixAt` appears in exactly two files — `Cloud.js` and `Sampler.js` — and **never in `Instances.js`.**

Under `<Instances>` there is no `setMatrixAt` call anywhere to intercept: not from app code, not from the library. The render is real — 200/200 slots written, `mesh.count = 200` — the wrapper captured all five args

perfectly, and routing is still zero.

Argument capture is scope-invariant and perfect in all three. Routing tracks one thing only: whether an H4-shaped destination hook exists.

7.5 Developer burden — H1's genuine win

A) manual tagging, `Trees.jsx` as it stands : 23 lines, per generator, per entity type
B) H1: refactor + wrapper : 1 line of wrapper ceremony
file total 274 -> 218 code 176 -> 159

23 lines of ceremony become 1, and the file shrinks 17 code lines. This is H1's strongest result.

The price is the refactor (+18/-13 for B) and what the developer must know: not "wrap your generator" but *where to draw the boundary* — A wraps the obvious function and captures none of the five values; B captures all five and required knowing what the record wanted before choosing the signature.

HMR: same as H2, not better. Records grew 200 → 400 → 600 monotonically with no invalidation marker; a module re-evaluation orphans the previous generation, which stays reachable from anything holding the old function reference.

StrictMode: the wrapper doubles (200 → 400) with no key to collapse on — two records with equal args and distinct `Matrix4` identities are indistinguishable from two genuinely co-located instances. The destination side doubles too, but 800 writes collapse to 400 distinct (`mesh`, `index`) pairs.

GroundCover: two wrappers required, not one. Eight parameters each to record four wanted values, and **seven locals had to be hoisted out** purely to keep RNG draw order intact — get that wrong and output diverges silently.

Verdict: best capture, zero routing. Needs something H4-shaped, exactly as H2 did.

8. H6 — The Composition

Premise: H4 routes but captures nothing; four hypotheses capture but cannot route. Chain them, using the `Matrix4` object as the join key. The untested link is `Matrix4.prototype.clone()` — H5's proxy sees `dummy.matrix`, the same object every iteration, so only the clone produces distinct identity.

8.1 The clone ratio is not 1:1

	<code>updateMatrix</code>	<code>Matrix4.clone</code>	Ratio	Histogram	Stable
Trees	200	400	2.0	<code>{2: 200}</code>	All 200
GroundCover	805	805	1.0	<code>{1: 805}</code>	All 805

Trees is 2:1 because `dummy.matrix.clone()` appears at both `Trees.jsx:193` and `:196`. The two clones are **distinct objects with identical bytes**, landing in different meshes. GroundCover splits `grass {1: 750}`, `bush {1: 55}` — no variation by entity type.

During the placement loop, zero clones came from any `Matrix4` other than the dummy's.

8.2 Adjacency is not the rule — receiver identity is

Pairing correctness, full payload: **Trees** byte-identical to ground truth (c569dd23...), 200/200 trunk, 198+2+0+0 foliage, 0 clones unplaced. **GroundCover** 805/805, 0 mismatched, 0 unplaced.

Interleaving across a full lifecycle:

placement loop	400
geometry + material build	0
InstancedMesh ctor + commit effect	0
updateMatrixWorld + boundingSphere	0
clones from any other Matrix4	0

Content comparison also works — all matrix states are distinct — but **content cannot separate Trees' two clones from each other**, since they are byte-identical. Only object identity can.

The decisive test injected one foreign `Matrix4.clone()` per instance:

naive_floorNover2_ruleStillHolds:	false
adjacencyPairingCorrect_afterFilteringByReceiverIdentity:	true
foreignClonesAlsoPairedToAnUpdateMatrix:	200

A foreign clone is a/so "the clone that follows an `updateMatrix`." Counting rules break outright. **Adjacency is only sound once the stream is filtered by receiver identity** — is [this](#) the dummy's matrix? That filter, not the ratio, is what makes the pairing work. An implementation shipping the counting rule would pass on this codebase and break on any code cloning a matrix mid-loop.

8.3 The end-to-end join

Join key is the object identity of the `Matrix4` returned by `clone()`: `clone.outId` → paired `updateMatrix` on one end, `setMatrixAt.matrixId` → (`mesh`, `index`) on the other.

	Intercepted	Ctor fill	App writes	Joined	Unjoined
Trees	800	400	400	400	0
GroundCover	1,610	805	805	805	0

Both rebuilt payloads byte-identical to ground truth. Trees across all three meshes; GroundCover across five meshes and both entity types, (`kind`, `variant`, `instanceId`) 805 matched / 0 missed.

`manualSourceRefsUsed`: 0.

The H4 limit is lifted. H4 could not recover global generation order on GroundCover, because the write stream arrives grouped by mesh. The clone carries the loop-time clock through the join:

h4 write-stream order:	g0g0g0g0g0g0g0g0g0g0g0... (grouped by mesh)
true generation order:	g0g1g1g1b1g1g2g0g2g0g1g2b0...
recovered from umSeq:	g0g1g1g1b1g1g2g0g2g0g1g2b0...
GLOBAL_ORDER_RECOVERED:	true

The grass/bush interleave is recovered. The clone chain supplies what H4's byte-join could not.

8.4 Blast radius — the expectation inverted

`Matrix4.prototype.clone()` was expected to be a noisy interception point. Measured on one real page load (React 19.2.4 StrictMode, three Three.js instances):

```

total Matrix4.clone() calls      2410    (= exactly 2 x 1205)
from three.js / drei / R3F / OrbitControls    0
reaching a setMatrixAt          1205
orphans from StrictMode's discarded render pass 1205

updateMatrix() calls, first read      7297
updateMatrix() calls, after camera activity 27606    (unbounded, per-frame)

```

Every clone call came from the application. Three.js uses `.copy()` on its matrix hot paths, not `.clone()` — confirmed across ten operations the app performs, all zero. Meanwhile `Object3D.updateMatrix()`, H5's interception point, fired 27,606 times for 1,205 instances and grows every frame.

The point expected to be dirty is quiet; the point already tested is the noisy one.

Overhead, median of seven trials after warmup:

	ns/call	Overhead	Slowdown
Unpatched	72.9	—	1.00×
Call-through only	88.4	+15.5	1.21×
Full record	1834.6	+1761.7	25.17×

For the 1,205 that matter: **2.12 ms** full record, 0.019 ms call-through, one-off at mount.

8.5 StrictMode and HMR

Live browser join: 1205/1205, zero unjoined, per-mesh 200/198/2 and 262/263/225/31/24. Payload hashes computed in the page match Node ground truth exactly.

StrictMode. Modelled as H1 established (double render, single host, double effect), the join produces 800 records collapsing to 400 distinct (`mesh`, `instanceId`) pairs with zero conflicting args. Critically, `recordsSourcedFromDiscardedRenderPass1: 0` — **the join is driven from the write side, so pass 1's 400 orphaned clones never enter it.** They are dropped for free. In the real app the render doubled but the commit effect ran once: 1,205 writes, 1,205 distinct pairs, no duplicates.

This is exactly the collapse key H1 identified as missing from loop-side capture. The destination side supplies it.

HMR, tested live with a comment-only edit to `Trees.jsx`, restored and hash-verified:

```

probe survived (Fast Refresh)      mesh uuids: 8 -> 11
new Trees mesh uuids                3
GroundCover meshes untouched        5
application writes                  1205 -> 1605
distinct (mesh, instanceId) pairs   1605    (equals writes, 0 collisions)

```

The clone record has no generation marker — matching H1 and H2 — but the destination side supplies the boundary. A remount rebuilding host objects stamps every record with a fresh mesh uuid; a remount reusing the host rewrites the same keys, so last-write-wins collapses it.

One residual hazard. If instance count shrinks on a reused mesh (200 → 150), slots 150–199 still hold generation-1 matrices and still render. Their join records are stale but indistinguishable from live ones by (`mesh`, `instanceId`) alone. A count-aware sweep is required.

9. Cross-Cutting Findings

9.1 The line that held five times

Every hypothesis that records before the destination exists fails to route. H3, H5, H2's loop-time captures, and H1 all sit on that side. H4 alone records at the destination, where mesh identity is the call's own receiver, and routes at 100% without deriving anything.

H1 is the cleanest demonstration because it isolates the variable: perfect argument capture, zero destination identity, in the same experiment.

9.2 Dangerous near-misses

Three mechanisms produce results that look correct under casual testing and are wrong:

- **H5's identity assumption** — group 0 is nearly identity, wrong 10.6% of the time at 200 trees and 30.6% at 260, while group 1 is 100% wrong.
- **H1's minified parameter names** — `["i", "n", "o", "e", "a"]` reads as names and carries no information.
- **H6's counting rule** — `clone N belongs to updateMatrix floor(N/k)` holds perfectly on this codebase and breaks outright the moment any foreign `Matrix4.clone()` occurs mid-loop. Only receiver-identity filtering makes the pairing sound.

All three would pass a demo. None would survive general use.

9.3 The symmetric traps

Trees' foliage meshes share one geometry and separate only by material. GroundCover's grass variants share one material and separate only by geometry. Any implementation keying on one of those alone works on one component and silently fails on the other.

9.4 The gap nothing closes

Under library-owned instancing, transforms go directly into a `Float32Array` and no `setMatrixAt` call exists anywhere in the program. H4 has nothing to attach to; every other hypothesis needs H4. This is not a scoping choice — it is an absence.

9.5 What is now tested

H6 ran the composition and it holds: 400/400 on Trees, 805/805 on GroundCover, 1205/1205 in the live browser, byte-identical to ground truth, zero manual annotation, plus recovery of global generation order.

Two expectations were overturned in the process. `Matrix4.prototype.clone()` was assumed noisy and proved to be the quiet interception point — 2410 calls on a real page load, every one from the application, zero from three.js, drei, R3F, or OrbitControls, because three.js uses `.copy()` on its matrix hot paths. `Object3D.updateMatrix()`, already patched in H5, is the noisy one at 27,606 calls and growing per frame.

What remains untested is not the mechanism but its generality: every measurement comes from two components in one codebase, both hand-rolled. The residual hazard identified in 8.5 — stale slots on a

shrinking reused mesh — has been characterised but not solved.

10. Summary Table

	H1 Wrapper	H2 Call-site	H3 Replay	H4 Interception	H5 Proxy	H6 Composition
Args recovered	200/200 named	200/200 positional	200/200 exact	none	200/200 exact	200/200 + 805/805
Routes	0/200	Scope-dependent	No	1205/1205	Provably no	1205/1205 live
Global generation order	No	No	N/A	No	No	Yes
Developer effort	1 line + refactor	Zero	Zero	Zero	Zero	Zero
Build tooling	No	Yes	No	No	No	No
Cost	Negligible	+2.24% bundle, 2.3M events	415 ms cold	0.14 ms/build	Negligible	2.12 ms/mount
HMR behaviour	Accumulates, no marker	Accumulates, no marker	N/A	Write-once, no replay	N/A	Mesh uuid bounds it
StrictMode	Doubles, no collapse key	Doubles loop-time	N/A	Collapses on (mesh, index)	N/A	Orphans dropped free
Works under drei <Instances>	Args yes, routing no	No	N/A	No hook exists	N/A	No

11. What the Evidence Supports

Three components are established well enough to build on:

Location stamping is solved by build-time `__source` injection. It has precedent, needs no experiment, and eliminates the hand-maintained `line:` integers that have drifted three times in this project.

Destination interception is H4 essentially as tested. Two constraints carry over: the probe must install at import time before the first commit, since writes are once-only with no replay; and it is process-global.

Non-instanced objects need almost nothing new. Construction is the destination, so location stamping plus the existing literal-patching engine covers Terrain and Water.

Per-instance argument capture is now established for hand-rolled instancing. H6's composition — transform capture, clone interception filtered by receiver identity, destination interception — produced complete records for both components with zero annotation, verified live. Two implementation constraints carry over: the pairing must filter by receiver identity rather than count, and a count-aware sweep is needed to retire stale slots when

an instance count shrinks on a reused mesh.

One component remains unsolved: **library-owned instancing**. Under drei `<Instances>` transforms go straight into a `Float32Array` and no `setMatrixAt` exists anywhere in the program. This is an absence, not a scoping choice, and no hypothesis addresses it.

The honest position: routing solved, location solved, per-instance capture solved for hand-rolled instancing and verified in a real browser — unsolved where a library owns the buffer. Those are two claims, and shipping should state which one it makes.